

See discussions, stats, and author profiles for this publication at: <https://www.researchgate.net/publication/311543519>

Parallel Neural Nets Using Pattern-Partitioning for Process Distribution by Means Of PVM

Article · December 2013

CITATION

1

READS

8

1 author:



Anurag Rana

Arni University

28 PUBLICATIONS 15 CITATIONS

SEE PROFILE

Some of the authors of this publication are also working on these related projects:



Karka Theory [View project](#)



Radial basis optimization using fruit fly (aFOA) [View project](#)



Parallel Neural Nets Using Pattern-Partitioning for Process Distribution by Means Of PVM

Anurag Rana

M.Tech. CSE

Arni University, India

Abstract: A neural network is an inherently parallel system where many, extremely simple, processing units work simultaneously in the same problem building up a computational device which possesses learning and generalization recognition abilities. Implementation of neural networks roughly involve at least three stages; design, training and testing. The second, being CPU intensive, is the one requiring most of the processing resources and depending on size and structure complexity the learning process can be extremely long. Thus, great effort has been done to develop parallel implementations intended for a reduction of learning time. Pattern partitioning is an approach to parallelism neural networks where the whole net is replicated in different processors and the weight changes owing to diverse training patterns are parallelized. This approach is the most suitable for a distributed architecture such as the one considered here.

Incoming process distribution, service aiming for improving distributed system performance facilitating dynamic load balancing. A Neural Network Device inserted into the kernel of a distributed system as an intelligent tool, allows achieving automatically distribution of execution requests under some predefined criteria based on resource availability and incoming process requirements. This paper dual proposal, shows firstly, some design and implementation insights to build a system where decision support for load distribution is based on a neural network device and secondly a distributed implementation to provide parallel learning of neural networks using a pattern partitioning approach. In second case, the parallelized approach for learning of back propagation neural networks are shown some performance results. This includes a comparison of recall and generalization abilities and speed-up when using a socket interface or PVM.

Keywords: Parallelized neural networks, back propagation, partitioning schemes, pattern partitioning, system architecture.

1. INTRODUCTION

The back propagation (BP) learning algorithm is one of the most popular learning algorithms, due to its efficiency and wide range of applications. BP can be parallelized through two partitioning schemes; either the network or the training pattern space is partitioned [9][14][15]. In network partitioning, the nodes and weights of the neural network are distributed among distinct processors, thus the computations for node activation, node errors and weight changes are parallelized. In pattern partitioning the whole neural network is replicated in different processors and the weight changes owing to distinct training patterns are parallelized. Last scheme is fit properly to run on local memory architectures and suitable for problems with a large set of training patterns. In this paper we only concentrate on a pattern partitioning approach to parallelize the learning process for transparent process distribution in a computer network with a new variant of the per-epoch-variant training regime. The variant called variable-epoch training regime consists in randomly assigning the number of epochs locally performed before any exchange takes place. Higher speed-up was the motivation of this new approach previously envisioned for a socket-based interface [5].

In the following sections alternative parallel approaches, supporting architectures, the application and results concerning speedup, recall and generalization capabilities when contrasted against the conventional sequential approach will be discussed.

2. BACK PROPAGATION PARALLEL LEARNING ALGORITHM

For parallelizing the BP learning algorithm, we mentioned two schemes:

In network partitioning each processor processes its corresponding process and therefore, during the propagation and adaptation phases the processors need to establish communication with each other. Since this interaction and exchange of data is frequently done, this scheme demands a fine granularity of parallelism. This type of parallelism is advantageous on a Shared Memory Architecture or Multiprocessor [20]. In pattern partitioning [1][10][12] a single program is replicated among processors and each computer will execute its personal copy of this program on different data patterns. Figure 1 shows that, pattern partitioning replicates the neural network structure (units, edges and associated weights) at each processor and then the partitioned training set is distributed among processors. Each processor performs the propagation and adaptation phases for the local set of patterns. Also, each processor accumulates the weight changes

produced by the local patterns, which afterward are broadcast to other processors for updating weight values. This is done by using Ts/P patterns where Ts is the size of the training set and P is the number of processors committed to the learning process. Weight changes are performed in parallel and then the corresponding accumulated weight change vectors are exchanged between processors. This scheme is suitable for problems with a large set of training patterns, by permitting a more coarse parallelism than the network partitioning scheme. This scheme fit properly to run on Message Passing Multi-computers or Multiprocessors [20].

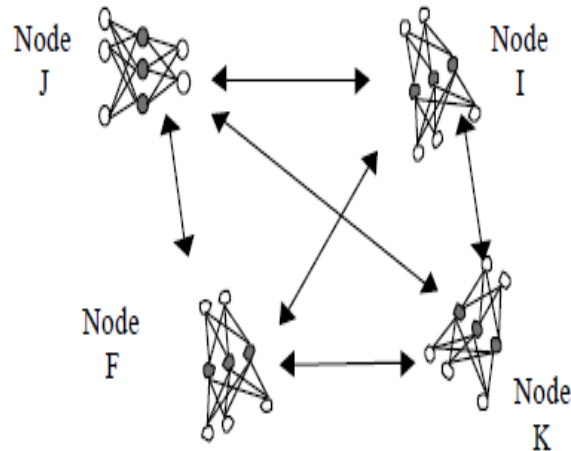


Fig. 1 Pattern Partitioning Scheme

Here distributed architecture supporting pattern partitioning for parallel training of neural networks propose. In the corresponding implementation the neural network is replicated in each system node where an individual learning process is running for the associated partition of the pattern space. Hence, weight changes are computed concurrently, exported, imported and adjusted accordingly until the whole parallel learning process is completed.

3. FOR PROCESS DISTRIBUTION USING NEURAL NETWORK DEVICE

Neural networks, an intelligent facility to automatically distribute in a computer network are the most appropriate node to user incoming process in accordance to its computing requirements [2].

The model assumes that:

- To improve response time for user processes for relevant performance.
- The network is formed by a set of N nodes, such that each of them can contribute with different performance to a user process depending on its demands.
- Every user incoming process comes to the network through an entry node, before passing to the execution node (see Fig. 2). Process behavior and resource requirements can be determined by a program profile file or explicitly declared by the incoming process.
- Processes coming to be served in this network have different demands on system resources (CPU, Memory and I/O devices).
- Evaluator module within the Operating System kernel evaluates process attributes, requirements and system state at the process arrival time.
- Using the output of the evaluator, as input, a module decides which node in the network can accomplish more efficiently the process execution and then process migration takes place.

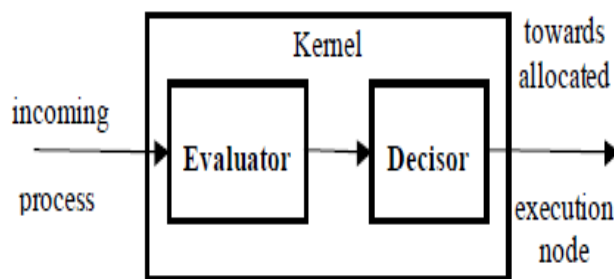


Fig. 2 Kernel portion

Let us assume the following scenario:

We have a system where N available nodes differ essentially in Current Memory Capacity (CMC) and MIPS provided. Due to system dynamics they also differ in Current Available Processing Power (CAPPw). User processes are CPU intensive process and their main requirements are Memory Required (MEMRq) and Desired Response Time (DRT).

CMC, CAPPw, MEMRq and DRT, are each divided into a number of levels (high, medium and low, or more levels). Other processes requirements on system resources, such as access to secondary storage, can be equally satisfied by any of the available nodes and there will not be network transfers (except for initial process migration, which we assume is equally costly for every node).

Then the following simple distribution criterion can be applied:

- Having MEMRq best fit satisfied, satisfy DRT by allocating the process to the best fitted node (the one with minimum CAPPw fulfilling process requirement). In case of equal CAPPw values for more than one node then node selection is random.
- If the strategy also considers the situation where idle nodes exists, then; if for two or more nodes CAPPw is equal, and some of these nodes is in idle state (IS) then the process is allocated to that idle node.

This second decision attempts to balance the workload.

For this allocation criterion, with N system nodes, 3+6N binary inputs will suffice to depict process requirements (3 bits) and system state (6 bits per system node), while the size of total pattern space is given by:

$$T = [32(N + 1) 2^N] - [3^N + 1 (22N + 2^N)]$$

Because only legal inputs conforms a training set for the neural net, the second term of T excludes the cases in which MEMRq is greater than CMC available.

4. IMPLEMENTATION OF PARALLEL LEARNING

4.1 ALGORITHM

The basic steps of a parallel backpropagation learning algorithm using variable-epoch regime are depicted below. We recall that, under these approaches, during a number (one or more) of epochs, the submission of all patterns in the partition, the corresponding computations and the accumulation of weight changes must be performed before weights update takes place, then the next epoch interval begin.

Parallel Training Algorithm

Repeat

1. For each pattern

1.1 Compute the output of units in the hidden layer.

1.2 Compute the output of units in the output layer

1.3 Compute error terms for the units in the output layer.

1.4 Compute error terms for the units in the hidden layer.

1.5 Compute weight changes in the output layer.

1.6 Compute weight changes in the hidden layer.

2. Exchange of accumulated weight vectors

2.1 If epoch interval was reached then sends local hidden weight vectors and output weight vectors.

2.2 Receive remote hidden weight vectors and output weight vectors. 2A node is defined as being in an idle state when no user process is running.

3. Update weights changes in the output layer.

4. Update weights changes in the hidden layer.

Until (current error < max. accept. err.) or (number of iterations = maximum number of iterations)

Here a parent process spawns several BP processes with the corresponding parameters.

Each process during the propagation phase, if epoch interval was reached, the accumulated weight change vector is broadcast to others processes and remote accumulated weight change vectors are received from other processes. The reception is not blocking, since if nothing has arrived, the children go ahead. Finally, each BP process performs the adaptation phase and completes one epoch.

Each child finish when the current error of the neural network is less than the maximum accepted error or the number of iterations is greater than the admissible number of iterations.

4.2 ALTERNATIVE SUPPORTS TO IMPLEMENTATION ON PVM

In previous works [5][6] a real implementation was built on the processors distributed in a LAN of workstations. Each process ran in a workstation. The routines used a socket interface as an abstraction of IPC (Inter process Communication) mechanism [3][4][16][17].

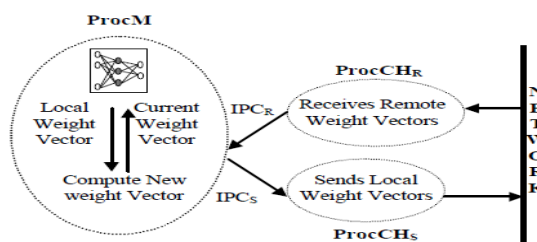


Fig. 3. System Architecture.

A UDP protocol was chosen because we were working in a reliable LAN and even, if a package missing happens the learning process is not be sensitively affected (each process will update the weights with the packages received). Figure 3 depicts the underlying system architecture and procedures supporting the parallel learning process for this approach. The current work with Parallel Virtual Machine (PVM) is discussed now. PVM is created to link computing resources and provide to the user with a parallel platform for running their computer applications, independent of the number of processors[18][19]. PVM supports a very efficient message- passing model.

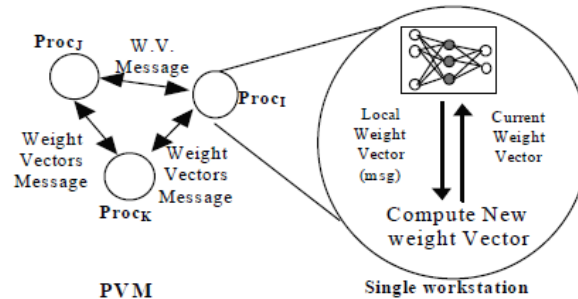


Fig. 4. PVM System Architecture

Figure 4 shows a substitute support to implement our particular application on Parallel Virtual Machine.

5. DESCRIPTION OF EXPERIMENTAL STUDY

Per-Epoch-Variant (PEVt) training regime only referred in experiment. The variable number of epochs locally performed before any exchange took place was chosen as a random number R between 1 and 10. Better results were observed for greater R values, but reported results correspond to average values.

Let be S the total pattern space. The processors training sets (ts) were subsets of S .

For sequential training, the training set T_s was built by uniform selection of $X\%$ of the pattern space S .

For parallel training, the pattern space was divided into n subsets and each subset was assigned to one processor (virtual or not) in parallel execution. The training subsets ts_i were built by uniform selection of $(X/n)\%$ of the pattern space S .

Values for X were chosen as 45 and 90.

In what follows the experiment identifiers indicate:

<Training type>-<size(%) of Training Set>/<number of subsets for parallel execution>

To compare results, the neural net was trained sequentially (i), in parallel using socket (ii) and in parallel using PVM (iii):

- Experiment **SBP-X/1**: SBP-45/1 and SBP-90/1. Size (T_s) = 45% and 90% of S respectively.
- Experiment **PBP-X/n**: PBP-45/2 and PBP-90/2. Three disjoint subsets of $(X/n)\%$ of S were selected and each subset was assigned to one processor in different workstations. Size (ts_i) = 10% and 20% of S respectively.
- Experiment **PVM-X/n**: PVM-45/2, PVM-45/4, PVM-90/2 and PVM-90/4. The software allows any numbers of processors to be created without any relationship to the number of real processors. In this state three and six disjoint subsets of $(X/n)\%$ of S were selected respectively, and each subset was assigned to one process. The number n of parallel processes was set to 2 and 4. Size (ts_i) = 10%, 5%, 20% and 10% of S respectively.

In two stages (learning and testing), the following parameters were used in each case:

- During Sequential Processing, T_s (the training set of the unique neural network) was used on the learning stage.
- During Parallel Processing, on the learning stages, ts_i was the local training subset submitted to the BP_i , with the accumulated weight changes vectors received from other BP_j networks (with training subset ts_j , $j \neq i$). For that reason, $T_s = ts_1 \cup ts_2 \cup ts_3 \cup \dots \cup ts_n$ was the training set for all BP_i networks at the learning stages.

In both cases T_s and S (the whole sample space) were used in the testing stage (for Recall and Generalization respectively).

Figure 5 shows an example of parallel partitioning scheme for experiments SBP-X/1, PBP- X/3 and PVM-X/3:

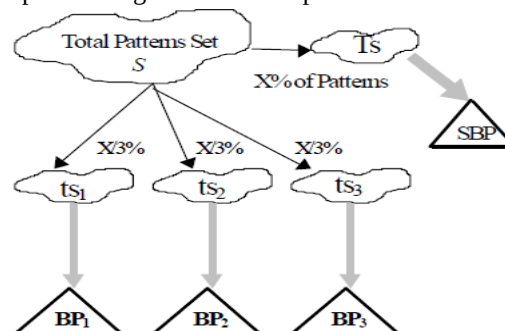


Fig. 5 Parallel Process Partitioning Approach.

During these processes, the following relevant performance variables were examined:

Training process:

L_T : Learning time, is the running time of the learning algorithm.

N_{iter} : Number of iterations needed to reach an acceptable error value while training.

Testing process:

$R = rcg/Size(Ts)$ is the recall ability of the neural network. Where rcg is the total number of patterns recognized when only patterns belonging to the Training Set (Ts) are presented, after learning, to the network. The objective is to analyses if each network can acquire the learning of other networks that were running in parallel with it.

$G = gnl/Size(S)$ is the combined recall and generalization ability. Where $Size(S)$ is the size of the Total Pattern Space and gnl is the total number of patterns recognized when all possible patterns are presented, after learning, to the network.

$Sp = L_{Tapproach1} / L_{Tapproach2}$ is the ratio between the learning times under different approaches (sequential or parallel).

$Rc_{B/C} = Sp/(R_{sq} - R_p)$ is the benefit-cost ratio for recall. It indicates the benefit of speeding up the learning process, which is paid by the cost of possibly loosing recall ability.

$Gn_{B/C} = Sp/(G_{sq} - G_p)$ is the benefit-cost ratio for generalization. It indicates the benefit of speeding up the learning process, which is paid by the cost of possibly loosing generalization ability.

6. RESULT

Neural network was trained sequentially (SBP), in parallel using socket (PBP) and in parallel using parallel virtual machine (PVM). The corresponding mean values of the performance variables are shown in the following figures and tables.

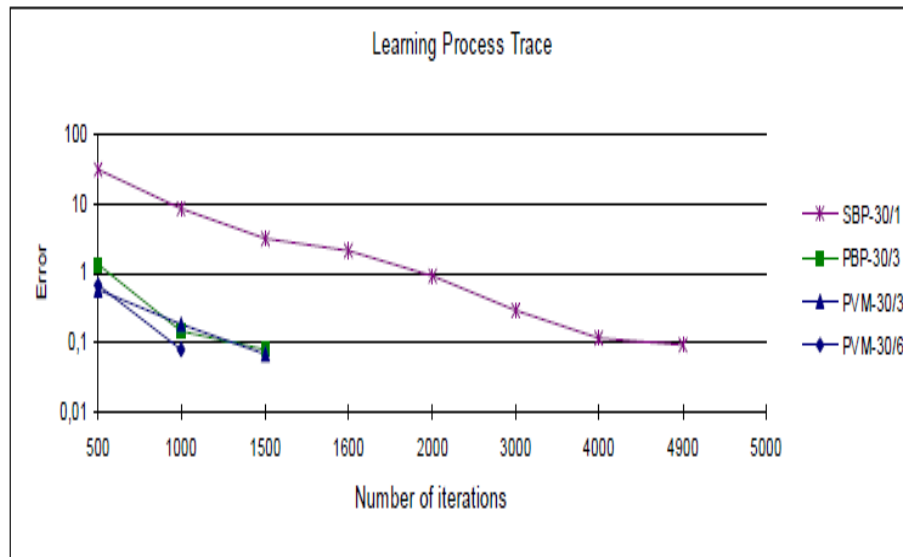


Fig. 6 - Number of iteration needed under sequential and parallel processing (best case)

Figure 6 observe a reduction greater than one third in the number of iteration needed to achieve permissible error values was achieved. Results for the partitioning scheme of 30% are shown but using either parallel partitioning approach attains similar results.

Table 1: Summary of L_T , R & G results.

Experiment	Learning Time	Recall %	Generalization %
SBP-45/1	1985	100	97
PBP-45/2	427.89	97-85	96.93
PVM-45/2	125.37	97.87	94.71
PVM-45/4	56.05	93.08	90
SBP-90/1	4567	100	98.5
PBP-90/2	1358.45	99.66	97.97
PVM-90/2	275.89	99.92	98.01
PVM-90/4	92.45	98.98	97.87

L_T expresses in seconds while R & G are expressed in percentile values.

Figures 7 and 8 show the associated loss in recall and generalization of the neural network for different sizes of the training set.

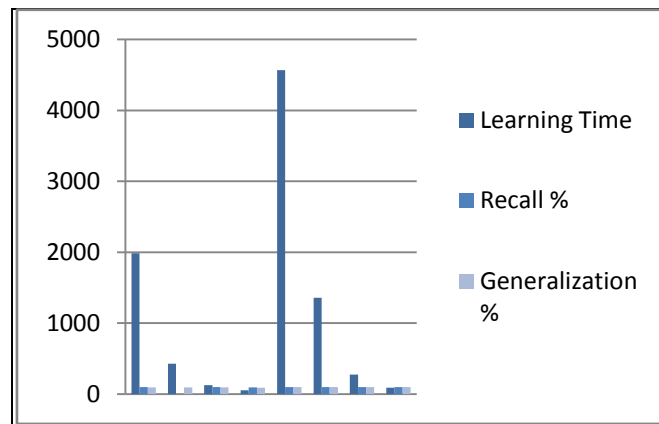


Fig. 7-Values of Recall and Generalization for Ts=45% of S

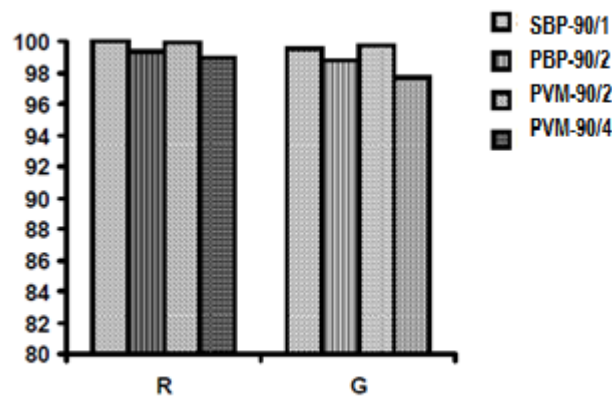


Fig.8- Values of Recall and Generalization for Ts=90% of S

The detriment of recall and generalization capabilities decreases as the training set size is incremented. Their values range from 0.07% (PVM-90/3) to 5.2% (PVM-45/4) for recall. In the case of generalization the values range from -0.13% (PVM-90/2) to 5% (PVM-45/4). Opposite to the expected, in this case, an improvement was also detected: PVM-90/2 achieved a generalization capability better than the sequential BP. Table 2 indicates the speed-up in learning time attained through parallel processing when different sizes of the portions of the total pattern space S are selected for training the neural network.

Table 2(a) shows the ratio between the sequential learning and the parallel learning times ($S_P = L_{T(S)} / L_{T(P)}$).

SBP-45/1 vs.			SBP-90/1 vs.		
PBP-45/2	PVM-45/2	PVM-90/4	PBP-90/2	PVM-90/2	PVM-90/4
4.64	15.83	35.41	3.362	16.55	49.4

Table 2(a) - Speed-up values achieved through parallel processing vs. sequential processing.

As the size of T_s increases (from 45% to 90%) then an increment of the speed-up can be observed under variable-epoch training. This effect shows a substantial improvement over the per-epoch approach used in earlier implementations. Moreover, increment of the speed-up can be observed among different parallel implementations.

Table 2(b) shows the ratio between both parallel learning times.
($S_{PVM} = L_{T(PBP)} / L_{T(PVM)}$).

PBP-45/2 Vs.		PBP-90/2 Vs.	
PVM-45/2	PVM-45/4	PVM-90/2	PVM-90/4
3.41	7.63	4.92	14.99

Table 2(b) - Speed-up values achieved through parallel processing with PVM vs. parallel processing with Socket

Both parallel implementations showed comparable capability, but PVM-X achieved a substantial increment in speed-up with values ranging from 3.41 to 14.99 times faster than PBP-X.

It is interesting to observe in table 3 the Benefit-Cost Ratio, which gives an indication of a speed-up S_p obtained at the cost of a detriment in recall or generalization. Table 3(a) shows for PVM-X, the benefit-cost ratio for recall ability.

$$R_{C/B/C} = S_p / (R_{sq} - R_{pPVM})$$

and table 3(b) shows the benefit-cost ratio for generalization ability

$$G_{B/C} = S_p / (G_{sq} - G_{pPVM}).$$

SBP-45/1 Vs PVM-45/2	SBP-45/1 Vs PVM-45/4	SBP-90/4 Vs PVM-90/2	SBP-90/4 Vs PVM-90/4
15.06	40.09	31.77	77.81

Table 3(a) - Benefit-Cost Ratio for Recall.

SBP-45/1 Vs PVM-45/2	SBP-45/1 Vs PVM-45/4	SBP-90/4 Vs PVM-90/2	SBP-90/4 Vs PVM-90/4
8.06	7.56	--	42.67

Table 3(b): Benefit-Cost Ratio for Generalization

All cases, it can be observed good ratios between benefits and costs. In the particular case of PVM-90/2 an increment of speed-up was simultaneously detected with an increment in generalization capability, hence the benefit cost ratio is not registered. This performance variable is of great help to inspect the goodness of a parallel design for training neural nets.

7. CONCLUSION

We discussed and showed results of an improved implementation using a Parallel Virtual Machine approach and a new variant called per-epoch-variant (PEVt) approach. Most recent results were contrasted against sequential and parallel approaches previously implemented. In the parallel case the PEVt and the per-epoch approaches were compared showing better performance for the new variant. Furthermore, we need to remark that PVM provided us a unified framework within which our parallel application was developed in an efficient and clear manner. That resulted in a straightforward program structure and very simple implementation. PVM transparently manipulated all message routing, synchronization aspects, data conversion, message packing and unpacking, process group manipulation and all aspect regarding heterogeneity. All these factors contributed to reduce development and debugging time. It worth remarking that a more effective implementation of parallel back propagation neural network was completed. Finally, at the light of the effectiveness showed by the distributed approach for the parallel learning process by means of PVM, at the present time, testing with larger number of processors and different training set sizes are being performed for different neural networks.

REFERENCES

- [1] Berman F., Snyder L. - On mapping parallel algorithms into parallel architectures- Parallel and Distributed Computing, pp 439-458, 1987.
- [2] Cena M., Crespo M. L., Gallard R. Transparent Remote Execution in LAHNOS by Means of a Neural Network Device. Operating System Reviews, Vol. 29, Nro 1, ACM Press, 1995.
- [3] Colouris G., Dollimore J., Kindberg T. -Distributed Systems: Concept and Design - Addison-Wesley, 1994.
- [4] Comer, D. E., Stevens, D. L. - Internetworking with TCP/IP - Vol. III - Prentice Hall.

- [5] Crespo M., Piccoli F., Printista M., Gallard R.- A Parallel Approach for Backpropagation Learning of Neural Networks- Proceedings of 3er Congreso Argentino de Ciencias de la Computación, Universidad Nacional de La Plata, Vol. 1., pp 145 – 156., September 1997.
- [6] Crespo M., Piccoli F., Printista M., Gallard R.- Parallel Shaping of Backpropagation Neural Networks in Workstations-Based Distributed Systems- Proceedings of International ICSC Symposium on Engineering of Intelligent Systems – EIS’ 98, University of La Laguna, Vol. 2., pp 334 – 340, Tenerife, Spain, February 1998.
- [7] Foster, Ian T. : Designing and Building Parallel Programs - Addison Wesley, 1995.
- [8] Freeman, J., Skapura, D. Neural Networks. Algorithms, Applications and Programming Techniques. Addison-Wesley, Reading, MA, 1991.
- [9] Girau, B. - Mapping Neural Network Back-Propagation onto Parallel Computers with Computation/Communication Overlapping. Proceedings of Euro Par’95.
- [10] Kumar, V., Shekhar, S., Amin, M.- A Scalable Parallel Formulation of the Backpropagation Algorithm for Hypercubes and Related Architectures. IEEE transactions on Parallel and Distributed Systems, Vol. 5. Nro.10, pp 1073 - 1090, October 1994.
- [11] McEntire, P. L., O’Reilly, J. G., Larson, R. E. (Editors) : Distributed Computing: Concepts and Implementations - Addison Wesley, 1984 .
- [12] Petrowski, A., Dreyfus, G., Girault, C.- Performance Analysis of a Pipelined Backpropagation Parallel Algorithm. IEEE. Transactions Networks, Vol. 4, pp 970 - 981, November 1993.
- [13] Plaut, D., Nowlan, S., Hinton, G. Experiments on Learning by Backpropagation. Tech. Report, CMU-CS-86-126, Carnegie Mellon University, Pittsburg, PA, 1986.
- [14] Rumelhart, D., Hinton, G., Willams, R. Learning Internal Representations by Error Propagation. MIT Press, Cambridge, Massachusetts, 1986.
- [15] Rumelhart, D., McClelland, J. Parallel Distributed Processing, vol. 1 y 2. MIT Press, Cambridge, MA, 1986.
- [16] Stevens, R.W.- Advanced Programming in the UNIX Environment- Addison-Wesley Publishing Company. 1992.
- [17] Stevens, R.W.- UNIX Network Programming. Prentice Hall-Englewood Cliff. 1990.
- [18] Sunderam, V., Manchek, R., Jiang, W., Dongara, J., Bengueulin, A., Geist, A. – PVM3 – User’ s Guide and Reference Manual – Oak Ridge National Laboratory: Tennessee, 1994.
- [19] Sunderam, V., Manchek, R., Jiang, W., Dongara, J., Bengueulin, A., Geist, A. – PVM: Parallel Virtual Machine – The MIT Press, Cambridge, Massachusetts, 1994.
- [20] Tanembaun, A.. Modern Operating Systems-Prentice Hall 1992.
- [21] V.Mani, S.Suresh and S.N.Omkar: Parallel Implementation of Back-Propagation Algorithm in Networks of Workstations, IEEE Transactions on parallel and distributed systems, Vol 16, No.1 January 2005.